Trabalho Prático 1 – Indexação Geoespacial e Geometria Computacional Híbrida

Gabriel dos Santos Nogueira – 2024088648
santosgabriel@dcc.ufmg.br

Pedro Augusto da Silva – 2024005661
pedroaugusto@dcc.ufmg.br

Vinícius Alves de Andrade – 2024005572
viniciusalves@dcc.ufmg.br

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

1 Introdução

Atualmente, com o advento das tecnologias, muitos restaurantes se beneficiam de aplicativos dedicados aos serviços de *delivery* – quando o cliente solicita um produto (nesse caso, comida) e o estabelecimento o entrega na casa de quem pede. Nesse contexto, é de extrema importância desenvolver mecanismos que identifiquem, com precisão, entregadores próximos a tais locais, uma vez que isso significa menores tempos de espera do cliente e mais fluxo de saída para o estabelecimento, um benefício para ambos os lados.

Esta documentação tem como objetivo explicar uma aplicação desenvolvida para lidar com a visualização e a escolha eficiente de entregadores no entorno de um restaurante e que estejam dentros de um raio informado pelo usuário. Para tanto, ela foi dividida em 9 seções, onde em cada uma será debatida um tópico sobre o desenvolvimento do algoritmo. Nesta seção, a 1, foi apresentada uma introdução do problema. A seção 2 discute premissas adotadas como ponto de partida. Em seguida, na seção 3, são explicitados os requisitos para o funcionamento da aplicação. A seção 4 mostra a organização dos diretórios. Depois, na seção 5, é análisada toda lógica estrutural do programa. Na seção 6 há a integração entre o usuário e o sistema. Em seguida, na seção 7, há a análise experimental as principais funções. Após isso, na seção 8 há o *link* dorepositório do projeto na plataforma *GitHub* e, por fim, as referências bibliográficas.

2 Premissas

A base de dados possuía certos valores duplicados. Para lidar com isso, bares com o mesmo tipo e nome da via, número, complemento, bairro e nome do bar foram considerados iguais. Todavia, ainda permaneceu alguns casos problemáticos, como por exemplo o bar Mascate: na base, ele possui o mesmo endereço, mas a duplicação ocorre pelo fato de em uma ocorrência estar registrado com o nome "RUNERIA MASCATE SAVASSI"e em outra com o nome "MASCATE SAVASSI". Esses dados representam um mesmo estabelecimento. Todavia, não foram removidos, uma vez que, se considerássemos bares com

o mesmo tipo e nome da via, número, complemento e bairro iguais, independentemente do seu nome fantasia, esabelecimentos em *shoppings centers* mal registrados na base (que não possuem a informação complemento, crucial para separar lojas nesses aglomerados), seriam considerados um mesmo estabelecimento, mesmo sendo lojas diferentes. Nesse sentido, entre ocultar bares e mostrar bares duplicados, o projeto optou por manter a segunda opção.

Também vale ressaltar que uma quantidade ínfima de bares (21 dos 20.859 da base) não tiveram suas coordenadas encontradas pelas consultas via *OSMnx*. Tendo em vista que o número representa uma porcentagem ínfima do total, esse fato não representou um grande problema.

3 Requisitos

Sobre o programa, ele foi testado no sistema operacional *Microsoft Windows 11 Home Single Language*, com processador *12th Gen Intel(R) Core(TM) i5-1235U* e 16 GB de memória RAM. Deve-se utilizar a linguagem Python na versão 3.12.3 para o bom funcionamento do programa. Além disso, algumas bibliotecas terceiras devem estar presentes no computador, como:

- **OSMnx (versão 2.1.1):** Responsável por toda a parte geoespacial. Ela simula a rede viária de Belo Horizonte por meio do `OpenStreetMap`. No código, ela é usada para carregar o grafo (representação da rede viária), projetar as coordenadas dos entregadores nas vias mais próximas e calcular a distância real percorrendo as ruas;
- **NetworkX (versão 3.7):** Fornece a estrutura de dados de grafo utilizada pelo *OSMnx*, uma vez que o grafo viário de BH é um objeto do *NetworkX* (`MultiDiGraph`). Ela também é usada para tratar exceções de caminho inexistente, garantindo que o programa não quebre se não houver rota entre dois pontos;
- **Shapely (versão 2.2.0):** Serve para a manipulação de geometrias (pontos, linhas, polígonos) no plano cartesiano. No projeto, ela é essencial para converter as coordenadas de latitude/longitude (graus) para o sistema métrico (metros) antes de usar algumas operações do *OSMnx*;
- **Dash (4.4.1):** Permite criar a interface *web* interativa em *Python*. Ele gerencia as páginas (`home.py` e `mapa.py`), os *callbacks* (quando o usuário clica em "Enviar!") e a troca de dados entre as telas através do `dcc.Store`;
- **Dash-Leaflet (1.1.3):** Integra mapas interativos baseados no *Leaflet* (do *OpenStreetMap*) dentro do *Dash*. Ela fornece os componentes visuais do mapa: `dl.Map`, `dl.TileLayer` (o fundo do mapa), `dl.CircleMarker` (para os entregadores e bares) e `dl.GeoJSON` (para agrupar os entregadores não capturados).

4 Arquitetura

Sobre a aplicação, o seu código é separado em diretórios para melhor organização do projeto. Assim, podemos destacar:

- `assets/`: Armazena os arquivos estáticos de estilização (como o arquivo `style.css`). O *framework Dash* carrega automaticamente o conteúdo desta pasta para personalizar a aparência visual do site;
- `cache/`: Diretório gerado automaticamente pela biblioteca *OSMnx*. Ele armazena

em cache as respostas das requisições HTTP feitas ao *OpenStreetMap* (como o *download* do grafo viário), otimizando o tempo de carregamento e evitando requisições repetidas e desnecessárias ao servidor;

- `dados/`: Contém os arquivos de entrada de dados do sistema. Inclui a base de dados dos bares (`bares_coordenadas.csv`) e o grafo viário de Belo Horizonte (`bh_drive.graphml`), que são a base para as buscas e cálculos de rotas;
- `pages/`: Armazena os módulos que definem as páginas da interface web utilizando o recurso *Dash Pages*. Contém o arquivo `home.py` (responsável pelo formulário de entrada de dados) e `mapa.py` (responsável por processar a busca e renderizar o mapa e a tabela de resultados);
- `src/`: É o núcleo lógico do projeto. Contém os módulos de *backend* separados da interface, como as classes de domínio (`bar.py` e `entregador.py`), a estrutura de dados (`triecompacta.py`) e as funções úteis de busca e de codificação *Geohash* (`auxiliares.py`).

5 Estruturas e Funções Lógicas

Desenvolvido no diretório `src`, o código do *backend* é separado em módulos para melhor organização. As principais estruturas de dados implementadas foram a Árvore TRIE Compacta (através da classe `TRIECompacta`) e os TADs `Coordenada`, `Bar` e `Entregador`, a fim de representar coordenadas, bares e entregadores numa aplicação. TADs secundários, como `_Noh` o e `_Endereco`, dão apoio para as representações da árvore de prefixos e dos bares. Além disso, para cada estrutura, foi criado om conjunto de métodos e funções para uma melhor manipulação da mesma. No diretório `src`, estão os arquivos `auxiliares.py`, `bar.py`, `entregador.py` e `triecomposta.py` (implementações das classes Coordenada, Bar, Entregador e Árvore Trie Compacta, bem como funções relacionadas.

Analisando separadamente cada módulo, podemos destacar:

1. `auxiliares`: módulo que implementa estruturas, constantes e funções que dão apoio a outras classes na aplicação.

   As principais constantes definidas são:

   - `LATMIN, LATMAX, LONGMIN` e `LONGMAX`: delimitam a área do globo planificado a ser analisada em latitude e longitude. Por padrão, possuem os valores -90, 90, -180 e 180, isto é, todo o globo terrestre.

   A estrutura implementada é:

   - `Coordenada`: representa um par de coordenadas. Assim, contém dois atributos: `lat` (latitude) e `long` (longitude). Será utilizada por classes como `_Endereco` – esta, utilizada apenas por `Bar` – e `Entregador`. Seu método principal é:
     - `geohash()`: retorna uma *string* que representa o código GeoHash de `NIVEIS` (por padrão, `NIVEIS` = 8) dígitos na base numérica de 32 algarismos da respectiva coordenada. Tal código é calculado da seguinte forma: dado um par de coordenadas a ser codificado, iniciamos com dois intervalos (listas) com

       `intervaloLong = [LONGMIN, LONGMAX]`
       `intervaloLat = [LATMIN, LATMAX]`.

       Como $32 = 2^5$, faz-se $5 \cdot$`NIVEIS` iterações, alterando entre longitude e la-

titude, verificando se a longitude (latitude) é maior ou igual que a média entre os dois valores de `intervaloLong` (`intervaloLat`). Se verdadeiro, então concatenamos o valor 1 à direita de um numero binario inicializado em 0, e atualizamos

```
intervaloLong[0] = (intervaloLong[0] + intervaloLong[1]) / 2
(intervaloLat[0] = (intervaloLat[0] + intervaloLat[1]) / 2).
```

Se falso, concatenamos um 0 também a direita, e fazemos

```
intervaloLong[1] = (intervaloLong[0] + intervaloLong[1]) / 2
(intervaloLat[1] = (intervaloLat[0] + intervaloLat[1]) / 2).
```

Para a próxima iteração, se a anterior analisava longitude (latitude), agora é analisado latitude (longitude), e o processo continua. A cada 5 iterações, convertemos o binário de 5 dígitos para um caractere da base 32, concatenamos tal caractere à direita de uma *string* inicialmente vazia e zeramos o binário para as próximas 5 iterações. `NIVEIS` indica a quantidade de caracteres no código, onde cada caractere é calculado por 5 iterações desse processo.

Ao final, faremos 2^x, com $x = \left\lfloor \frac{\texttt{NIVEIS} \cdot 5 + 1}{2} \right\rfloor$, partições em longitudes e 2^y, com $y = \left\lfloor \frac{\texttt{NIVEIS} \cdot 5}{2} \right\rfloor$, partições em latitude, uma vez o algoritmo começa iniciando a análise em longitude e, para cada caractere, há um número ímpar de iterações. No programa em questão, como $\texttt{NIVEIS} = 8$ e considerando que a Terra possui cerca de 40.075 quilômetros de largura na linha do Equador e $20.037,5$ quilômetros de "altura", dividimos o globo em $2^{x+y} = 2^{\texttt{NIVEIS} \cdot 5}$ células, cada uma contendo, aproximadamente, $\frac{40.075.000}{2^x} = 38,2$ e $\frac{20.037.500}{2^y} = 19,11$ metros de largura e de altura, aproximadamente. Vale ressaltar que a largura varia de acordo com a latitude envolvida, uma vez que latitudes maiores em módulo representam faixas de larguras menores na Terra. Assi, concluímos que a precisão de todo o código Geohash é um retângulo de dimensões $38,2m \times 19,11m$.

Na execução do programa, são utilizados dois parâmetros: uma *string* `prefixo` e um número positivo `raio`. O valor de `raio` determina circunferências no entorno dos bares (de raio igual ao numeral) cujos nomes possuem como prefixo o valor da *string*, para que os entregadores nelas inscritos sejam capturados, isto é, estão a menos de `raio` metros de distância dos bares. Para implementar essa lógica, as seguintes funções foram desenvolvidas:

- `_niveis_para_raio(raioMetros)`: um código *Geohash* de 8 caracteres cobre uma célula retangular de área pequena $(38,2m \times 19,11m)$. Essa é a célula mais especializada. Entretanto, o valor do diâmetro definido por `raioMetros` pode ser maior do que as dimensões dessa célula, então uma circunferência de raio `raioMetros` não pode ser inscrita num retângulo com as mesmas dimensões. Nesse caso, devemos agrupar células menores e formar uma célula maior, até encontrar uma com dimensões capazes de envolver a circunferência de raio dado. Os z primeiros caracteres de um código *Geohash* representam um código *Geohash* de z níveis (obtigo agrupando-se $32^{\texttt{NIVEIS}-z}$ células especializadas).

  Cada caractere do código representa 32 divisões do plano. Assim, primeiro

analisamos 1 (32^0) célula (a mais especializada), e testamos se ela cobre uma circunferência determinada pelo raio. Se sim, então temos uma boa aproximação para as coordenadas. Se não, agrupamos 32 (32^1) células especializadas e repetimos o processo. Fazemos o mesmo até o limite, com um agrupamento de $32^{\texttt{NIVEIS}}$ células especializadas.

- `_dimensao_celula(niveis):` retorna as dimensões da célula (em graus) resultado de um código *Geohash* de `niveis` níveis. Aqui, as dimensões de longitude e latitude, em graus, são, respectivamente, $\frac{\texttt{LONGMAX} - \texttt{LONGMIN}}{2^x}$, com $x = \left\lfloor \frac{\texttt{niveis} \cdot 5 + 1}{2} \right\rfloor$, e $\frac{\texttt{LATMAX} - \texttt{LATMIN}}{2^y}$, com $y = \left\lfloor \frac{\texttt{niveis} \cdot 5}{2} \right\rfloor$.
- `_geohashes_vizinhos(coordenada, niveis):` dados um par de coordenadas e um inteiro representando o nível da codificação *Geohash*, a função retorna os códigos *Geohash* de nível conforme parametrizado para a célula central que contém as coordenadas informadas e das células imedatamente vizinhas, isto é, aquelas nas posições norte, nordeste, leste, sudeste, sul, sudoeste, oeste e noroeste em relação à central. Isso é útil porque, caso um bar tenha coordenadas próximas às bordas de uma célula, por mais que o diâmetro caiba nela, o centro da circunferência não estará no centro da célula, então o raio pode "invadir"células vizinhas. Nesse caso, temos que analisar, também, todas as células ao redor da central. Isso é feito deslocando as coordenadas passadas em valores proporcionais às dimensões da célula nos sentidos dos eixos de longitude e latitude.
- `busca_entregadores(bares, trieEntregadores, raio):` dada uma lista de bares, para cada bar, analisa o menor conjunto de células que podem conter entregadores que estão a uma distância de `raio` metros do bar, utilizando, para tanto, as funções vistas acima. Por fim, retorna uma lista de tuplas de três elementos, contendo um bar, um entregador próximo e a distância real dele ao bar.
- `busca_inteligente(trieBar, trieEnder, prefixoBar, prefixoEndereco):` busca os prefixos `prefixoBar` e `prefixoEnder` nas TRIEs Compactas `trieBar` e `trieEnder` e retorna os bares cujos prefixos de nome e de endereço satisfazem tais parâmetros.
- `cria_tabela(lista, velocidade):` recebe como parâmetros uma lista de tuplas de três elementos: bar, entregador e a distância do entregador ao bar (pelas vias) e um valor numeral representando a velocidade (em metros por segundo). Retorna uma lista de tuplas de quatro elementos: nome do bar, chave do entregador, distancia do entregador ao bar (pelas vias) e tempo esperado gasto do entregador até chegar ao bar (pelas vias).

2. `bar`: módulo que implementa a classe Bar e funções que dão apoio a essa classe na aplicação.

   As estruturas e classes implementadas são:

   - `_Endereco:` estrutura simples utilizada por Bar que armazena dados de seu endereço, como o tipo da via, o nome da via, o número, o complemento, o bairro e suas coordenadas.
   - `Bar:` classe que codifica um bar, armazenando seu nome e seu endereço. Seus

principais métodos são:

- `nome()`: retorna o nome do bar.
- `coordenada()`, `latitude()` e `longitude()`: retorna um par de coordenadas, latitude e longitude, ou apenas um deles.
- `geohash()`: retorna o código *Geohash* de suas coordenadas.

A função presente nesse módulo é:

- `le_bares(caminho)`: recebe um caminho de um arquivo *.csv* com informações de bares e retorna duas listas: uma com os bares cujas informações são válidas e outra contendo os números das linhas cujas informações não são capazes de inicializar bares.

3. `entregador`: módulo que implementa a classe Entregador e funções que dão apoio a essa classe na aplicação.
   - `Entregador`: classe que codifica um entregador. Seus atributos são: `chave`, uma `string` numérica representando um identificador único para cada entregador, e uma estrutura `Coordenada`, representando um par de coordenadas. Seus métodos mais importantes são:
     - `chave()`: retorna a chave (identificador) do entregador.
     - `coordenada()`, `latitude()` e `longitude()`: retorna um par de coordenadas, latitude e longitude, ou apenas um deles.
     - `geohash()`: retorna o código *Geohash* de suas coordenadas.
     - `distancia_reta(coordenada)`: recebe um par de coordenadas e retorna a distância, em metros, do entregador até o ponto das coordenadas, em linha reta.
     - `distancia_real(coordenada, Gproj)`: recebe um par de coordenadas e retorno a distância, em metros, do entregador até o ponto das coordenadas, utilizando as vias representada pelo grafo `Gproj`.

   As funções implementadas são:
   - `gera_entregadores(bares, N, sigma, maxTentativas)`: dada uma lista de bares e um valor inteiro `N`, gera `N` entregadores posiciados num plano de coordenadas. Para tanto, escolhe-se um bar aleatório na lista dos bares e, depois, tenta plotar um par de coordenadas válidas (dentro dos intervalos de latitude e longitude permitidos) seguindo uma distribuição normal com desvio padrão no valor de `sigma`. Faz esse processo até conseguir uma plotagem válida ou até realizar `maxTentativas` tentativas. Nesse último caso, plota o entregador exatamente no bar.
   - `encontra_vias_proximas(entregadores, G)`: dado um conjunto de entregadores (plotados ou não em vias urbanas), mudam suas coordenadas para o ponto mais próximo e uma via. A função utiliza internamente a função `nearest_edges(G, X, Y)` do módulo `ox.distance`, otimizada para receber lista de coordenadas, uma vez que ela constrói uma estrutura espacial (*k-d tree*) sobre as arestas do grafo uma única vez e, a partir dela, resolve a consulta de vizinho mais próximo para todos os pontos em uma só passada. Caso fosse um método e realiza-se a consulta individualmente para cada entregador, tal estrutura seria construída e destruída várias vezes.

4. `triecompacta`: módulo que implementa a estrutura de dados TRIE Compacta e seus métodos.

   As principais estruturas são:

   - `_Noh`:

     Seus atributos são:

     - `rotulo`: armazena a sequência de caracteres (*string*) associada à aresta que chega até o nó. Na raiz da árvore, o atributo é inicializado como uma *string* vazia ();
     - `filhos`: dicionário que mapeia o primeiro caractere de cada ramificação filha para o seu respectivo objeto de `_Noh`, permitindo acesso direto em tempo constante;
     - `valores`: lista que armazena os objetos (como instâncias de `Entregador` ou `Bar`) associados à chave a qual o término ocorre no nó, possibilitando armazenar múltiplos registros que tenham a mesma chave sem sobrescrever.

     Seu método é:

     - `eh_folha()`: retorna valor booleano sinalizando se o nó é uma folha, sendo `True` caso o dicionário `filhos` esteja vazio e `False` caso tenha descendentes.

   - `TRIECompacta`: classe responsável por gerenciar toda a árvore de prefixos compacta. Seu único atributo é a `raiz`, que guarda o nó inicial onde a árvore começa a ser construída.

     Seus métodos são:

     - `_coleta(noh, resultado)`: método recursivo auxiliar que realiza uma busca em profundidade (*DFS*) a partir de um nó dado. Ele adiciona todos os elementos contidos na lista `valores` do nó à lista de `resultados` e percorre recursivamente todos os nós presentes no mapa de `filhos`;
     - `insere(palavra, valor)`: insere uma chave textual (`palavra`) e seu objeto associado (`valor`) na árvore. Se houver uma diferença no caminho, o método divide o nó em dois: cria um nó com a parte que é igual e abre novos caminhos para o restante; já se a palavra for exatamente igual a um nó que já existe, apenas guarda o valor na lista dele;
     - `busca_prefixo(prefixo)`: recebe uma cadeia de caracteres de consulta e percorre a árvore verificando cada caractere do prefixo. Se a correspondência for concluída com sucesso, chama o método interno `_coleta` a partir daquele ponto para retornar uma lista com todas as informações guardadas naquele ramo da árvore, e, caso não encontre o prefixo, retorna uma lista vazia.

6 Integração Entre Usuário e Sistema

Dada as estruturas e funções lógicas de apoio, foi possível montar o *backend* da aplicação, isto é, a lógica de processamento do código. Agora, é preciso realizar a integração dos dados fornecidos pelo usuário com o ferramental básico desenvolvido, aliando a isso uma interface visual (*frontend*) esteticamente agradável e funcional.

O desenvolvimento foi fundamentado na biblioteca *Dash* (*Python*), utilizando o padrão de arquitetura *Multi-Page Apps* (aplicações com múltiplas páginas). A navegação do sistema baseia-se na alteração de URLs (rotas), gerenciadas dinamicamente pelo *Dash*, que indicam em qual etapa do fluxo o usuário se encontra. Para garantir a consistência visual, o projeto conta com um arquivo de estilização customizado (`style.css`) aplicado por meio de classes CSS nos componentes.

Para viabilizar essa arquitetura, o sistema foi dividido em três camadas principais:

1. `app.py` **(Layout Principal):**
   Inicializa o servidor e define a "moldura"do site, servindo como ponto de partida do programa.
   - **Gerenciamento de Estado:** O arquivo instancia um componente global `dcc.Store(id='dados-usuario')`. Ele funciona como um armazenamento em memória (um dicionário temporário) que permite transitar os dados inseridos pelo usuário entre as diferentes páginas sem a necessidade de recarregar o navegador.
   - **Navegação:** Utiliza o `dcc.Location` para monitorar e alterar a URL e constrói um menu de navegação dinâmico varrendo o `dash.page_registry.values()`.
   - **Renderização:** O componente `dash.page_container` atua como o espaço reservado exato onde o conteúdo das páginas filhas (`home.py` ou `mapa.py`) é injetado dinamicamente conforme a rota acessada.
2. `pages/home.py` **(Interface de Entrada):**
   É a página inicial de interação, construída com componentes visuais (`html.Div`, `dcc.Input`) organizados em formato de *cards* e estilizados via classes CSS.
   - **Captura de Dados:** Contém três campos de entrada (*inputs*) para coletar os parâmetros da consulta: número de entregadores, nome (ou prefixo) do bar e o raio de busca em metros.
   - **O Callback (Função Reativa):** O grande destaque é a função `salvar_e_redirecionar`. Ela é acionada pelo *input* de cliques do botão "Enviar"(`n_clicks`), mas lê os valores dos campos apenas no momento do clique através de `State`. Utilizando o parâmetro `prevent_initial_call=True`, a função impede disparos acidentais ao carregar a página.
   - **Ação:** Se os dados estiverem válidos, a função empacota-os em um dicionário, salva no `dcc.Store` (primeiro *output*) e altera a URL para `/mapa` (segundo *output*), redirecionando o usuário.
3. `pages/mapa.py` **(Interface de Resultados e Lógica):**
   É a página onde a lógica de processamento pesado (o *backend* geográfico) é executada e os resultados são renderizados.
   - **Processamento e Busca:** O *callback* `executar_main` é acionado automaticamente ao detectar dados no `dcc.Store`. O código reconstrói a base de bares via arquivo CSV, gera os entregadores nas coordenadas delimitadas, projeta-os nas vias reais usando a biblioteca `osmnx` e realiza buscas espaciais.
   - **Geração Visual:** A página renderiza uma interface rica contendo:
     - Um **Mapa interativo** (`dash_leaflet`) com pinos coloridos customizados

em HTML para os bares, marcadores em círculo para os entregadores capturados e um *cluster* (agrupamento dinâmico) para os entregadores não capturados.

– Uma **Legenda flutuante** gerada dinamicamente que associa as cores dos pinos aos nomes dos bares.

– Uma **Tabela dinâmica** que exibe o nome do bar, a chave do entregador, a distância real calculada e o tempo estimado (calculado dividindo a distância pela velocidade padrão de 8 m/s), todos formatados e ordenados.

7 Análise Exploratória

Para esta análise com o objetivo de determinar o quão viável é o aplicativo e quais são os seus limites de tempo, foi variado principalmente o raio geográfico da busca, e o número total de entregadores que estão na frota (denominado N), visando entender como eles reagem em diferentes cenários.

Os mecanismos de tempo foram coletados a partir de uma função `time.perf_counter()`, método que avalia o tempo em precisão de milissegundos, desconsiderando tempos de renderização e focando no tempo de execução dos algoritmos de armazenamento e busca em si.

Os testes foram estruturados em alguns tópicos principais, sendo eles:

- O impacto isolado da variação do raio com uma frota fixa de entregadores;
- O impacto isolado da variação da frota com um raio fixo;
- O impacto de componentes internos na busca;
- O impacto da implementação de uma busca em células vizinhas para casos extremos;
- E, por fim, o impacto da variação de todas variáveis em conjunto.

Experimento 1:

O primeiro experimento visa avaliar o efeito gerado pelo tamanho do raio da área geográfica de busca na resposta do algoritmo de localização de entregadores. O parâmetro fixo aqui é a frota N de entregadores, que foi fixado em $N = 10$ mil componentes que foram distribuídos pela malha da cidade. Os raios foram variados e avaliados em seis diferentes ordens de grandeza, sendo elas: $9,5$ m, 50 m, 300 m, $2,4$ km, 9 km e 40 km (casos mais extremos). O tempo foi medido utilizando o método de contagem temporal a partir da função interna `busca_entregadores`; o tempo se inicia antes do *Geohash* ter seus níveis calculados e encerra quando os possíveis candidatos a entregar forem validados.

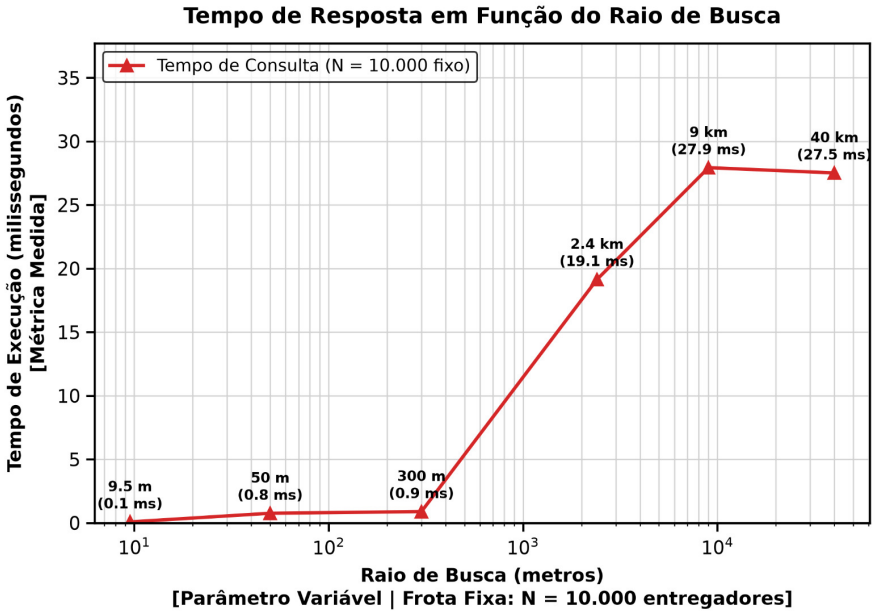


Figura 1: Tempo de Resposta em Função do Raio de Busca.

A evolução do gráfico da Figura 1 revela três regiões bem definidas de mudança, em que, para raios curtos a médios o tempo de execução é muito baixo, variando menos de um milissegundo, porque a função `_niveis_para_raio` seleciona três níveis mais profundos de *geohash*, e, como as caixas vizinhas cobrem uma área mais curta, a *trie* compacta descarta praticamente toda a árvore e retorna um conjunto pequeno de candidatos para validar.

A segunda região, dos 300 m a 9 km, representa o limite de tempo do algoritmo, pois a medida que o raio avança para uma escala de região, o tempo sobe para aproximadamente 28 milissegundos, o que ocorre porque para um raio maior, o *geohash* é reduzido para prefixos curtos, resultando em células maiores. Com isso a busca de consulta na *trie* retorna um número alto de candidatos. A terceira região e última do gráfico representa a saturação do algoritmo, em que a curva se estabiliza em cerca de 27 milissegundos.

Experimento 2:

A análise do segundo experimento é focada na comparação de desempenho da *trie*, e no quanto há um ganho real quando comparada com a abordagem mais simples possível, que é o cálculo da distância até o bar de cada entregador da cidade, um por um. Nesta análise o raio é fixado e apenas os entregadores variam, com números de 1000 a 40000.

O índice da *trie* foi calculado usando o método de tempo na chamada da função `busca_entregadores`. O índice temporal da força bruta, que é a linha vermelha, foi registrado percorrendo uma lista com todos os entregadores que foram gerados, executando a função de distância em linha reta para cada um deles (função `distancia_reta`).

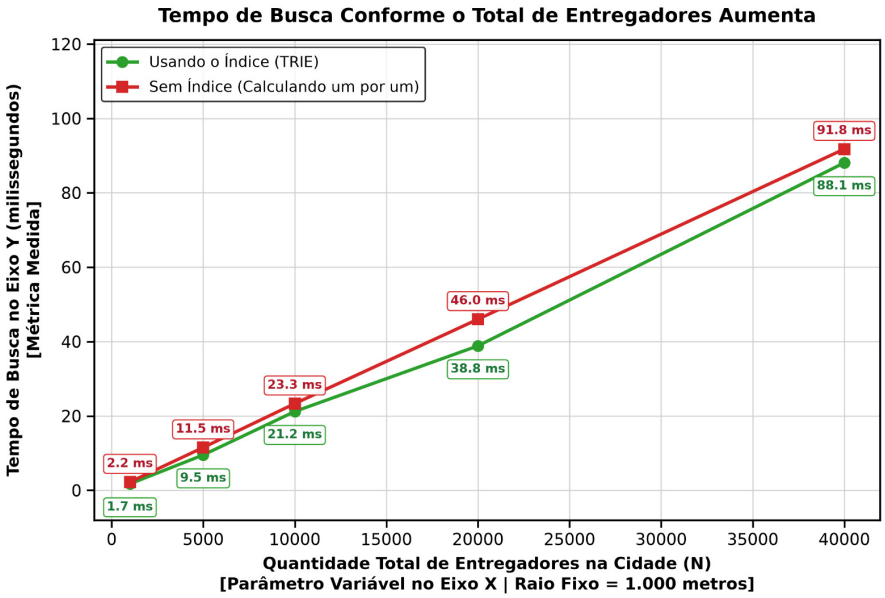


Figura 2: Tempo de Busca Conforme o Total de Entregadores Aumenta.

O gráfico da Figura 2 revela que há uma proporcionalidade de variação entre o tempo de busca e o número de entregadores, que a medida que aumenta, tem o tempo relativamente proporcional aumentado. O que ocorre pois as operações feitas pelo computador são proporcionais ao número de entregadores, se a frota dobra, o número de operações também dobra. É possível notar também que a estrutura da *trie* compacta é sempre mais rápida, como o *GeoHash* divide o espaço em blocos, se um entregador está em algum extremo da cidade, seu prefixo irá diferir logo nos caracteres iniciais, ignorando ramos de busca que não tem relação com a procura que está sendo feita. A economia de tempo aqui é de quase 16%.

Ao final as linhas acabam se aproximando porque mesmo dividindo as regiões em quadrantes e eliminando ramificações desnecessárias, a densidade de busca de entregadores no método da *trie* acaba continuando alta, e o custo matemático volta a pesar.

Experimento 3:

Para averiguar internamente, dentro da função `busca_entregadores`, qual o ponto crítico das etapas da função, e qual etapa tem o processamento mais rápido, foi aplicado o método de contagem temporal em três etapas internas diferentes da busca. São elas:

- `_geohashes_vizinhos`: com o tempo gasto para calcular coordenadas das células que cercam o bar;
- `busca_prefixo`: com o tempo gasto ao descer pelos nós da árvore para buscar entregadores;
- `distancia_reta`: com o tempo gasto para calcular a distância em linha reta de cada entregador.

Como parâmetros, foi fixada a frota de entregadores, novamente, em $N = 1000$, e o raio foi variado com as mesmas medidas (em metros) dos experimentos anteriores.

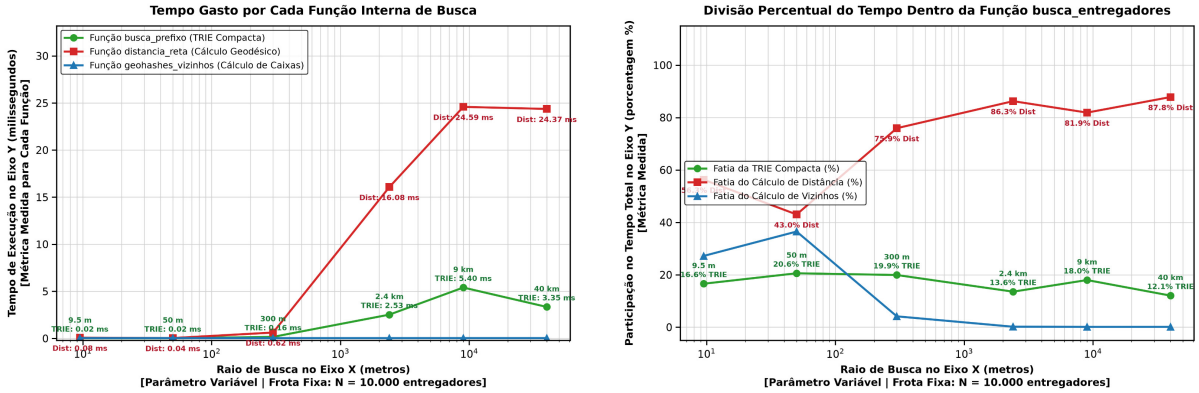


Figura 3: Tempo Gasto por Cada Função Interna de Busca (à esquerda) e Divisão Percentual do Tempo Dentro da Função `busca_entregadores` (à direita).

Os gráficos da Figura 3 revelam que o tempo é verdadeiramente gasto, em sua maioria, com o cálculo da distância em linha reta, o qual para raios curtos, como há um grande descarte de entregadores, é necessário apenas calcular os poucos que sobraram. Para raios mais longos com maior abrangência de células *Geohash*, executar repetidamente operações trigonométricas para cada entregador é o que faz o custo de busca ser alto.

O deslocamento na *trie* compacta é relativamente veloz, e, mesmo em raios maiores a partir dos 9 km, onde um número muito maior de entregadores é computado, o tempo é curto (mesmo percentualmente, seu tempo continua baixo e estável em relação ao todo). Ou seja, a árvore de prefixos cumpre o seu papel e realmente filtra os dados de maneira rápida. O cálculo de códigos de *Geohashes* de vizinhos é direto e não tem impacto significativo na conta total (estável e próximo de 0).

Experimento 4:

Este experimento justifica o uso da função `_geohashes_vizinhos`, em que, caso um estabelecimento esteja na extremidade de um retângulo espacial do *GeoHash*, não haja um efeito de borda que oculte entregadores mais próximos passíveis de busca. O que varia é o raio, e a frota de entregadores é fixa em $N = 1$ mil.

A linha vermelha no gráfico representa uma busca ingênua que considera somente a célula/retângulo onde o bar está, e a linha verde, o algoritmo completo implementado, que considera a existência de casos de borda, recolhendo os dados das 9 células adjacentes a partir da *trie*. O eixo Y mede a quantia de componentes (N) dentro de uma circunferência medida.

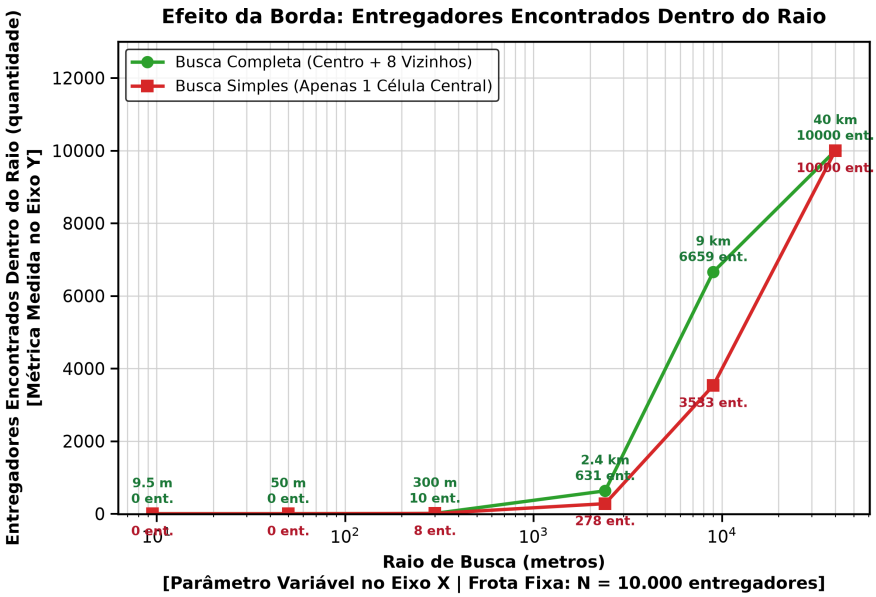


Figura 4: Efeito da Borda: Entregadores Encontrados Dentro do Raio.

Nota-se, pela Figura 4 que para raios muito pequenos não houve detecção de entregadores para nenhum dos dois modos, mas, que em raios maiores existe uma diferença grande válida entre ambos os métodos, e muitos entregadores não seriam contabilizados caso o método mais simples fosse utilizado. Ao expandir a busca o sistema acerta ao notar que o espaço geográfico não acaba na borda da coordenada da célula do meio. No fim, em um raio muito grande ambos os métodos convergem pois uma única célula já engloba o município.

Experimento 5:

Para abordar o efeito de variar ambos os parâmetros centrais do problema juntos, tanto o raio como o número N de entregadores, foi produzida uma matriz de riscos, que torna fácil visualizar onde no algoritmo o desempenho piora. Aqui, N varia entre 1000 e 40000, e o raio varia de $9,5$ m a 40 km. O método de medição continua sendo na função `busca_entregadores`.

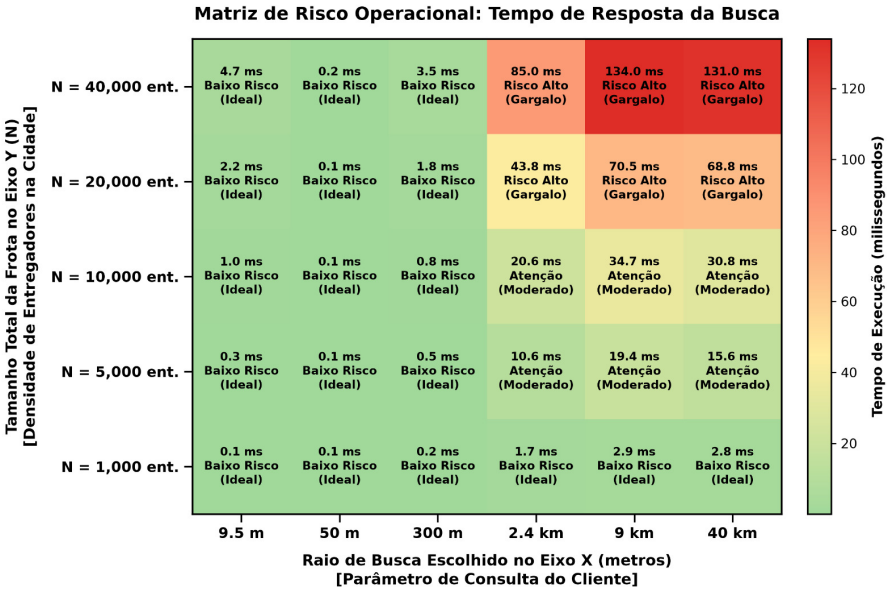


Figura 5: Matriz de Risco Operacional: Tempo de Resposta da Busca.

Os resultados na Figura 5 mostram que raios curtos em qualquer frota, e, o inverso, frotas pequenas com qualquer raio tem boa resposta do algoritmo, já que as células do *Geohash* são bem menores, e a cidade quase por completa é descartada, restando um N baixo. Em um raio maior, de $2,4$ km e em frotas de 5 a 10 mil componentes o algoritmo já é levemente pior em relação ao tempo pois a árvore deve devolver centenas de candidatos a mais, e as áreas cobertas pelas caixas *GeoHash* são maiores, aumentando o tempo de execução e busca.

O gargalo mesmo ocorre em faixas de N entre 20 e 40 mil e raios de $2,4$ km a 40 km, que é onde o sistema se esforça mais, existe um número muito expressivo de entregadores e o raio é composto por Belo Horizonte quase inteira. É portanto, possível concluir que em condições normais de uma entrega local, que compõe raios de tamanho médio como os realizados nos testes, o sistema funciona bem, e, em condições extremas, apesar de comparativamente mais lentas, a procura ainda é rápida.

8 Conclusão

Este trabalho teve como objetivo o desenvolvimento de uma aplicação capaz de organizar e visualizar, de forma eficiente, entregadores no entorno de estabelecimentos comerciais em Belo Horizonte. Para tanto, foi-se necessário integrar conceitos de geometria computacional, codificaçãoo espacial e desenvolvimento *web*, resultando em um sistema interativo funcional e com bom desempenho.

A base da solução foi a implementação da estrutura de dados *TRIE Compacta*, que se mostrou eficiente na busca de prefixos, seja para os nomes e endereços dos bares, seja para os códigos *Geohash* dos entregadores. O uso do *Geohash* proporcionou uma indexação espacial inteligente e eficaz, pois permitiu traduzir coordenadas geográficas em chaves textuais, simplificando a busca de entregadores próximos a um ponto dado.

A integração da interface *web*, por meio da biblioteca *Dash* e do componente *Dash-Leaflet*, permitiu que toda a lógica do *backend* geográfico fosse abstraída e simplificada em uma interface visual leve ao usuário. O mapa interativo, a legenda dinâmica e a

tabela ordenada por tempo de chegada deixam as informações mais claras e objetivas para o utilizador da aplicação. A arquitetura multi-páginas adotada manteve o código organizado e facilitou a separação entre a coleta de dados e o processamento dos resultados.

Por fim, a análise revelou que o algoritmo é robusto para os cenários esperados de uma operação de *delivery* real. Para raios de busca de tamanho médio (até alguns quilômetros) e frotas de até 10 mil entregadores, o tempo de resposta das consultas é de poucos milissegundos. O sistema é debilitado em condições extremas (raios que cobrem a cidade inteira e frotas altas de 40 mil entregadores), mas, ainda assim, mantém um tempo de processamento aceitável. A solução também soube lidar corretamente com o problema de borda das células do *Geohash* através da busca nas 8 células vizinhas, garantindo que entregadores próximos não fossem ignorados.

Feito isso, o projeto cumpre os objetivos propostos, oferecendo uma solução baseada em estruturas de dados complexas e eficientes juntamente com uma interface agradável e de fácil utilizacção, otimizando, assim, o problema de otimização logística em serviços de entrega.

9 *Link* do Repositório no *GitHub*

O projeto pode ser acessado no seguinte *link*:

- `https://github.com/Pedraugustos/TP1---ALG-2`

Referências

[1] BOEING, Geoff. **OSMnx: Getting Started**. 2024. Disponível em: `https://osmnx.readthedocs.io/en/stable/getting-started.html`. Acesso em: 20 out. 2026.

[2] **Dash Leaflet Documentation**. Disponível em: `https://www.dash-leaflet.com/`. Acesso em: 20 out. 2026.

[3] SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Upper Saddle River, NJ: Addison-Wesley, 2011.